

Reviewer Pack — Minimal Galois Group Action on Tensor Products and Boolean C...

Entry 9b076a6a3dcf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 13:53:26 UTC

Minimal Galois Group Action on Tensor Products and Boolean Circuit Monotone Width

Entry ID: 9b076a6a3dcf **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 13:53:26 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Galois Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For any given boolean circuit C with monotone width $w(C)$, the minimal order of the Galois group action on the tensor product of the field extensions corresponding to input and output variables of C is $O(w(C))$.

Rationale (proposer's reasoning):

Galois theory has been used to study symmetries in algebraic structures, but its application to boolean circuits is rare. This conjecture proposes that the minimal Galois group action on the tensor product of field extensions can be used as an invariant for monotone width. If true, it could provide a new perspective on circuit complexity and potentially lead to improved lower bounds.

Taxonomy category: minimal_galois_group_action (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0202f286e90877a0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal Galois group action order, when compared to the known monotone width, must be within a factor of 2 from the monotone width.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n):
        if n == 1:
            return [(0,)], 1
        else:
            subcircuits = [generate_boolean_circuit(random.randint(2, n//2)) for _ in range(2)]
            gate = (random.choice(['AND', 'OR']), subcircuits[0][0], subcircuits[1][0])
            return (gate, max(subcircuits[0][1], subcircuits[1][1]) + 1)

    def compute_monotone_width(circuit):
        gate, depth = circuit
        if gate[0] == 'AND' or gate[0] == 'OR':
            width = 0
            for input in gate[2]:
                width += max(len(input) for input in input)
            return width + depth - 1
        else:
            return 1

    def compute_galois_group_order(circuit):
        # Placeholder function to simulate computing the Galois group order
        # This is a dummy implementation and should be replaced with actual computation
        gate, _ = circuit
        if gate[0] == 'AND' or gate[0] == 'OR':
            return 2 ** (len(gate[1]) - 1)
        else:
            return 1

    n = random.randint(5, 40)
    circuit = generate_boolean_circuit(n)
    monotone_width = compute_monotone_width(circuit)
    galois_group_order = compute_galois_group_order(circuit)
```

```

if galois_group_order <= 0:
    return {
        "metric_name": "Galois Group Order",
        "metric_value": galois_group_order,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "galois_group_order_non_positive"
    }

expected_bound = monotone_width * 2
if abs(galois_group_order - monotone_width) > expected_bound:
    return {
        "metric_name": "Galois Group Order",
        "metric_value": galois_group_order,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": f"Order {galois_group_order} is not within a factor of 2 from monotone"
    }

return {
    "metric_name": "Galois Group Order",
    "metric_value": galois_group_order,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        mean_value = sum(result["metric_value"] for result in results if result["conjecture_holds"]) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if result["conjecture_holds"]) / len(results))
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    print(f"RESULT: {'SUPPORTED' if all(result['conjecture_holds'] for result in results) else 'FAILED'}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ddae8d6f.py", line 73, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ddae8d6f.py", line 45, in run_trial
    monotone_width = compute_monotone_width(circuit)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ddae8d6f.py", line 35, in compute_monot
    width += max(len(input) for input in gate[1])
               ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ddae8d6f.py", line 35, in <genexpr>
    width += max(len(input) for input in gate[1])
               ~~~~~~
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture's claim. | next: Re-run the test with proper error handling to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11474
2	propose	ollama_remote	glm4:latest	0	0	15093
3	preregistration	ollama_remote	glm4:latest	0	0	9759
4	novelty	ollama_remote	glm4:latest	0	0	8579
5	novelty	ollama_remote	glm4:latest	0	0	8481
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18522
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8886
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9387
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11934
10	judge	ollama_remote	glm4:latest	0	0	11756

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113871 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/9b076a6a3dcf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9b076a6a3dcf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9b076a6a3dcf.tar.gz (if generated)